A RISCV 32 bits Forth in 454 bytes

Alvaro G. S. Barcellos,
may 2026
github.com/agsb

To master riding bicycles you must ride bicycles

Proposal

A minimal functional Forth (engine and words)

Tuned for size (not for performance)

Expands the dictionary as any Forth

Fine tunned Read, Eval, Print, Loop

Use of Minimal Indirect Thread Code

Current result https://github.com/agsb/milliForth-RiscV

454 bytes (without ELF headers) using:

88 bytes with headers, (11 Forth minimal words)

66 bytes with native syscalls, (linux)

300 bytes of native code (warp and beat)

no numbers, no names, no terminal input buffer, no multitask, no multiuser, no rush

why do count without ELF headers ?
because barebones don’t uses

System details

riscv64-linux-gnu- (*gcc, *as, *ld, *objdump, *objcopy, *readelf)

-nostartfiles -nodefaultlibs -static -Oz -march=rv32gc -mabi=ilp32

qemu-riscv32

-display none -bios none -serial mon:stdio -smp 1 -machine virt

Could be any barebone system with _init, _getc, _putc, _exit functions

Know to tune RISCV native code

A RISCV have 32 registers of 32 bits, X0 to X31, and 32 bits opcodes.

X0 hardwire to 0x0, X1 return address (link and jump), X2 stack pointer,

Intensive use Standart Compressed Instruction Set Extension, (3 : 1)

Use X8-X15 make 16 bits opcodes, others make 32 bits opcodes,

No advantage in use SP as Forth stack, no magic automatic push or pull,

Use a register as constant memory pointer, and use relative offsets,

Use JAL and JALR with return address register,

Move values between registers, instead use of stack to memory.

Look Ma, no terminal input buffer !

Chuck executes or compiles each word individually rather than line by line. In fact Chuck doesn't really have lines. [Jeff Fox, 2000]

No Terminal Input Buffer, only a input stream parser

A Forth engine is not an editor, no need do copy, paste, delete, insert, etc

Simply parse the tokens, valid ASCII characters between spaces,

and convert tokens into hashes.

How to be without names

WE STI.. DON.. SEE THE NEE. FOR 31 CHA..... NAM.. IN THE GEN.... CAS.

[Chuck Moore, 1983] we still don't see the need for 31 characters names in the general case

Do only hashes, no names, on linked list headers of dictionary

Link + *Hash*, no name length limit. Collisions ? maybe 1 : 64k

Use 32 bits DJB2 hash, public domain, non cryptographic

(KEY R> DUP DUP + DUP + DUP + DUP + DUP + + XOR >R)

In all words of Forth 2012 annex H, only F~ and D< have same djb2 hash

Minimal world of words

Which could be the minimum set of words? It's a recurring theme.

My choices :

1. Words that can not be composed by another words

   NAND|NOR PLUS FETCH STORE NOTZEROIF

   NOTZEROIF return TRUE if TOS is not zero

2. Words that must exists to compile new words

   KEY EMIT COLON SEMIS EXIT USERAT

   USERAT return the address of user area variables

Warp and Beat ~ 256 bytes

Engines, mostly linear native code or two level calls

main cold warm miss abort quit	\ setup	~ 56
warp (token find eval compile execute)	\ R.E.P.L.	~ 42
token (skip hash scan mask)	\ parser	~ 46
beat (unnest next pick jump nest move)	\ M.I.T.C.	~ 32
comma _init _getc _putc _exit	\ support	~ 80

Minimal vocabulary ~198 bytes

0# TOS = zero if FALSE else TRUE then	U@ pointer to SP RP LATEST DP STATE
@ fetch, load a word from memory	! store, store a word into memory
+ plus, add two values	NAND NOT AND two values
: colon, change to compile mode, etc	; semis, change to execute mode, etc
KEY read a byte from system stream	EMIT write a byte into system stream
EXIT ends a word and go to next word	

~ average of (8 + 10) bytes per word

Extra vocabulary ~ 202 bytes

This extra set can be selected during native code compilation.

NAN Not a Number 0x80000000 to TOS	;$ DOCODE jump and link to ROS
LSHIFT left shift n bits	RSHIFT right shift n bits
ABORT restart the outer loop warp	BYE ends the Forth
. DOT shows a hex number at TOS	$ DOLAR receive a hex number to TOS

$ used to parse next token as 32-bit hexadecimal value, Eg. $ FEEDCAFE ,

More 42 primitives ~ 670 bytes

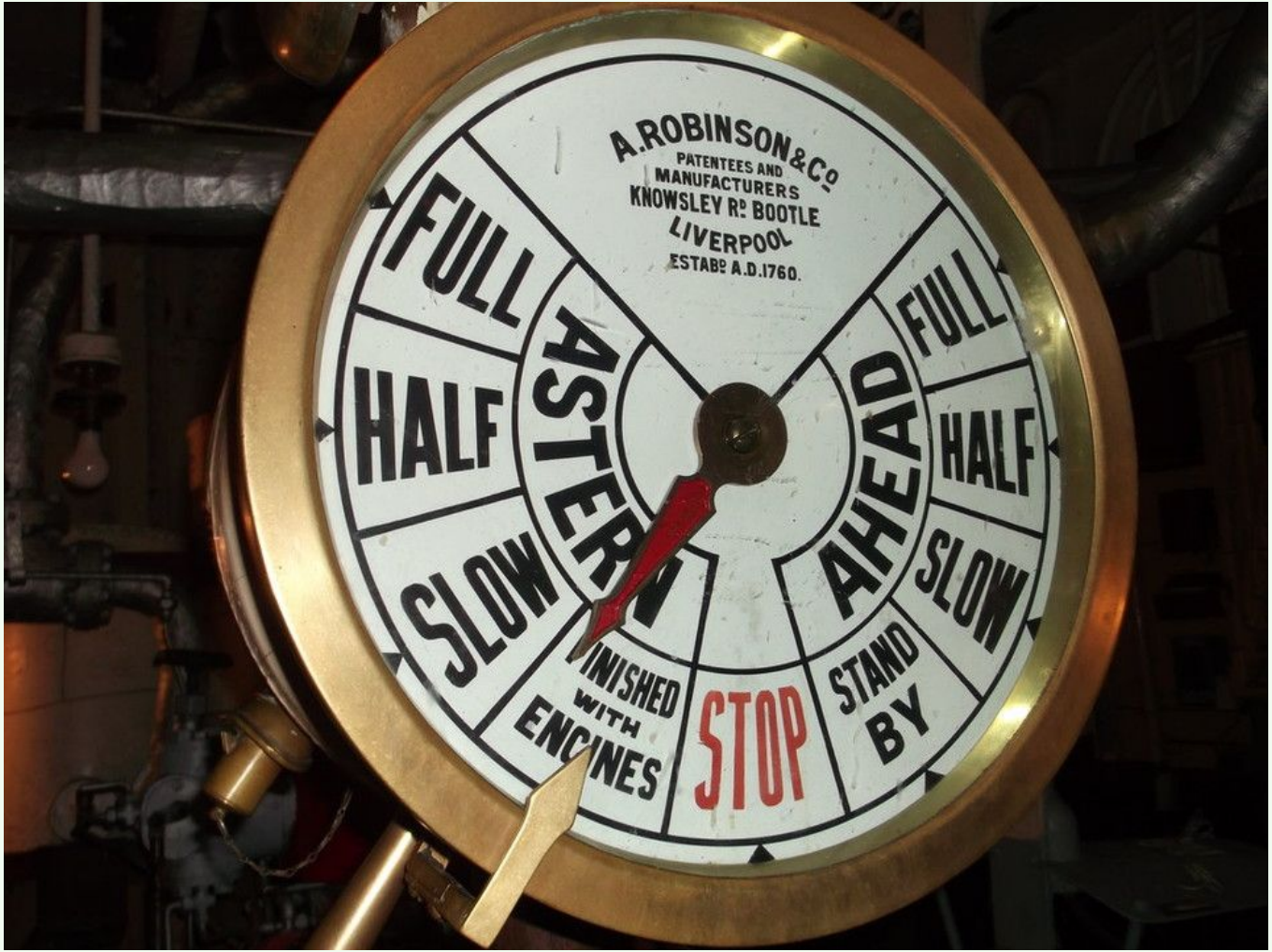
This set can be selected during native code compilation.

DUP	DROP	OVER	SWAP	ROT	LIT
AND	OR	XOR	NOT	NEG	ALIGN
R>	>R	R@	-	M*	/MOD
=	<	U<	0>	C@	C!
TRUE	FALSE	CELL	BRANCH	0BRANCH	GBRANCH
STATE	LATEST	HEAP	CEIL	FAUX	PERFORM
S0	R0	SP@	RP@	SP!	RP!

What is need for

Forth internal variables in user area (U@)

SP	the pointer to user stack
RP	the pointer to return stack
HEAP	the address of next unused memory cell, aka DP
LAST	the address of last node of dictionary linked list, aka LATEST
STAT	the multi STATE of Forth, (execute, compile, [postpone, etc])
FAUX	the address of last HEAP before start compiling, for RECURSE
CEIL	the address of last unused memory cell


A.ROBINSON&Co
PATENTEES AND
MANUFACTURERS
KNOWSLEY Rd BOOTLE
LIVERPOOL
ESTABD A.D.1760.
FULL
HALF
SLOW
ASTERN
FINISHED
WITH
ENGINES
STOP
STAND
BY
SLOW
HALF
FULL
AHEAD

Details

Only one flag, for immediate, 0x80000000. It also is NaN for numbers.

All BRANCHs are relative. Separated stacks of 36 (?) cells deep.

The IP (s1) and USER (s0) pointers must be keep reserved.

A hash is 4 bytes, while the average name length in Forth is 6 bytes.

Any word could start a compiled word. (no mandatory docol, dolit, etc)

Forth-83 defaults, FALSE is 0x00000000, TRUE is 0xFFFFFFFF

Could be native multi STATE. (execute, compile, [postpone, more ?])

RAM_LIMIT = 0x40000 (64k words of 4 bytes, may be far more than this)

Work in Progress

Most of the slowness because variables are accessed by offsets.

Create a speed version (aeForth) with more primitives words.

Compose (from various sources) the compiled word dictionary.

Test (really) on a Raspberry Pi RP 2350 RISC-V.

Create a version for Raspberry Pi RP 2040 ARM Cortex-M0.

Test (really) on a Raspberry Pi RP 2040 ARM Cortex-M0.

That’s all folks

References

https://groups.google.com/g/comp.lang.forth/c/NS2icrCj1jQ

https://archive.org/details/the-road-towards-a-minimal-forth-architecture

https://www.forth.org/fig-forth/fig-forth_PDP-11.pdf

https://rwmj.wordpress.com/2010/08/07/jonesforth-git-repository/

https://github.com/cesarblum/sectorforth

https://github.com/fuzzballcat/milliForth

https://github.com/agsb/milliForth-6502

https://github.com/agsb/milliForth-RiscV

References

https://en.wikipedia.org/wiki/RISC-V

https://theforth.net/package/minimal

https://github.com/larsbrinkhoff/nybbleForth

https://github.com/meithecatte/miniforth/tree/post1

https://github.com/larsbrinkhoff/nybbleForth

https://www.youtube.com/watch?v=Ok-1E8xwo90

https://www.greenarraychips.com/home/documents/greg/DB001-171107-F18A.pdf

Tuned for size not performance

What could be the minimal functional Forth engine and and primitive words ?

- FIGForth PDP-11 (16 bits, KA11/KD11, 1980) ~ 245 words, ? bytes
- jonesForth (16 bits, X86) ~ 85 words, ? bytes, ELF
- sectorForth (16 bits, x86) ~ 17 words, 512 bytes, ELF
- milliForth (16bits, x86) ~13 words, 336 bytes, ELF
- nybbleForth (16bits, verilog) ~12 words, ? , ?
- milliForth (8 bits, 6502) ~ 11 words, 1110 bytes, barebone
- milliForth (32 bits, RISCV) ~ 11 words, 454 bytes, barebone

Jump and Link (32 bits)

Jump to subroutine

; push RA, one deep

SP ← SP − 4

[SP] ← RA

; link and jump

RA ← PC + 4

PC ← EA

Return from subroutine

; pull RA, one deep

RA ← [SP]

SP ← SP + 4

; link and jump

ZR ← PC

PC ← RA

PC, program counter; EA, efective code address; the registers defaults to ZR (x0), wired zero; RA (x1), return address; SP (x2), stack pointer;

Details of indirect access

Use of: LW a4, SPTR (s0); LW a3, 0 (a4); ADDI a4, a4, 4; SW a4, SPTR (s0);

Instead of: LW a3, a4; ADDI a4, a4, 4; Because can not expose registers.

Then, all variables SP, RP, LATEST, DP, STATE are relatives to user struct U@

```
: SP = U@ ;        : SP@ SP @ @ ;        : SP!  SP  ! ;

: SP+ SP@ CELL + SP! ;     : SP-  SP@ CELL - SP! ;

: LATEST U@ CELL + CELL + ;    : LATEST@ LATEST @ ;    : LATEST! LATEST ! ;

: DP U@ CELL+ CELL + CELL + ;    : STATE U@ CELL + CELL + CELL + CELL + ;
```

#D14734 #E78570 #CCD6FB #6C8FB5

#614947 #B6685B #FDA369 #FDD6B6

#AD8997 #5B3A57 #E95223 #FC952D

#473B06 #DD9F0D #7DD6EA #E1F8FF

Made with dopely.top @dopelyapp

#E68F59 #073E6B #1973AE #A1D8EC

#333C31 #97B45D #A49752 #D7B56B

#0189DD #04BCF2 #8AD9F7 #D2B178

#F11532 #F046A1 #FA615B #F97D03

Made with dopely.top @dopelyapp

#4F566E #8C9EB4 #F79B7E #FFECA7

#7B574B #D3855F #FEBF72 #FFF0B6

#3C1C0F #855615 #AD972F #3B4D0C

#36352F #465321 #B59226 #DF560E

Made with dopely.top @dopelyapp

STOP
DEAD SLOW
SLOW
HALF
ASTERN
DEAD SLOW
SLOW
HALF
AHEAD